01 - Model records before querying

Explained answers and feedback

Identify keys, relationships and a reason to separate tables.

Textbook: database-foundations

A

Member PK member_id; Loan PK loan_id; Loan FK member_id. One Member has 0..many Loans; one Loan has exactly 1 Member. Cleo illustrates optional participation for Member.

B

Member 9 does not exist, so the foreign-key insert fails when enforcement is enabled. Repeated names could be updated inconsistently. member_id determines name; loan_id determines loan details. Move member facts to Member.

Exit / feedback

Repeated Loan.member_id is valid for multiple loans. Repeated loan_id violates uniqueness. If keys are confused, identify which entity each identifier names.

02 - Select columns and specify order

Explained answers and feedback

Read SELECT, FROM and ORDER BY before adding conditions.

Textbook: sql-basics

A

SELECT name, member_id FROM Member ORDER BY name DESC; returns ('Cleo',3), ('Bo',2), ('Ari',1).

B

SELECT loan_id, days FROM Loan ORDER BY days DESC, loan_id ASC; returns (102,5),(104,5),(101,3),(103,2). DISTINCT days ordered ascending returns 2,3,5. Secondary order makes equal-day results explicit.

Exit / feedback

SELECT reads; it does not change stored rows. Output columns come from SELECT; order comes from ORDER BY. If rows are missing, check whether an unintended filter was added.

03 - Filter rows and test boundaries

Explained answers and feedback

Apply WHERE with inclusive boundaries and explicit Boolean grouping.

Textbook: sql-basics

A

WHERE item = 'Camera' OR returned = 0 returns all four loans. AND instead returns no rows: Camera loans are returned. Retain ORDER BY loan_id.

B

WHERE returned = 0 AND days BETWEEN 2 AND 4 returns no current rows. Hypothetical days 1 excludes; 2 and 4 include. Include returned = 0 in each test.

Exit / feedback

item LIKE 'C%' matches Camera. Use IS NULL for missing values: = NULL is not true, and WHERE retains only true conditions. Loan columns in this dataset are NOT NULL; use a hypothetical nullable column for this explanation.

04 - Join related records without losing meaning

Explained answers and feedback

Match keys in an INNER JOIN and predict missing matches.

Textbook: database-solution

A

Use the worked join plus WHERE Loan.returned = 0. Returns Ari/102/Headset and Bo/104/Tripod. ORDER BY Loan.loan_id.

B

Wrong join gives 0 rows because 1,2,3 do not equal 101..104. All pairs give 3 * 4 = 12 rows. Join relationships using corresponding keys.

Exit / extension

Ari and Bo each appear twice; row count 4 is not distinct-member count 2. LEFT JOIN preserves Cleo with NULL loan columns. LEFT JOIN is a later extension, not a prerequisite for the independent INNER JOIN task.

05 - Aggregate first, then filter groups

Explained answers and feedback

Distinguish row filters, grouping and aggregate results.

Textbook: sql-basics

A

All rows: Camera/2/3, Headset/1/5, Tripod/1/5. With WHERE returned = 0: Headset/1/5, Tripod/1/5. WHERE removes source rows before grouping.

B

SELECT member_id, SUM(days) FROM Loan GROUP BY member_id HAVING SUM(days) >= 8 ORDER BY member_id; returns (1,8). HAVING filters groups; WHERE cannot filter on an aggregate over the group at that stage.

Exit / feedback

The preserved Cleo row contains NULL Loan.loan_id. COUNT(column) ignores NULL; COUNT(*) counts that row. If totals are multiplied, inspect join cardinality before changing aggregate functions.

06 - Make controlled changes and prove the result

Explained answers and feedback

Use a fresh copy, specific conditions and rollback while practising changes.

Textbook: database-solution

A

BEGIN; INSERT INTO Member VALUES (4,'Dev'); INSERT INTO Loan VALUES (105,4,'Light',1,0); SELECT * FROM Loan WHERE loan_id=105; DELETE FROM Loan WHERE loan_id=105; SELECT * FROM Member WHERE member_id=4; ROLLBACK; Final original counts: Member 3, Loan 4. Member 4 exists inside the transaction after the delete, but not after rollback.

B

CREATE TABLE Tag(tag_id INTEGER PRIMARY KEY, label TEXT NOT NULL); NULL label fails. INSERT INTO Tag VALUES (1,'Light'); succeeds if the table is fresh. Primary key ensures unique tag IDs; NOT NULL requires a label value, but does not itself prohibit empty text.

Feedback / exit

Retain requirement, authorised scope, preview, expected/actual affected rows and verification/recovery evidence. A repeated successful insert with the same primary key fails. If too many rows change, rollback and repair WHERE before retrying.